

MID-TERM GROUP PROJECT

Rwanda Road Safety Intelligence System

HDFS + Apache Spark (PySpark) Analytics Engine

Technical Report

Group 11

AUCA Masters — Big Data Essentials

September 2026

Data Source: Kaggle Road Accident Dataset (UK), used as a surrogate for the proposed Rwanda Road Safety Intelligence System.

Table of Contents

Table of Contents	2
1. Introduction and Case Study	4
1.1 Rwanda National Context (NISR Statistical Yearbook 2024)	4
1.2 Objective	4
2. Dataset	4
2.1 Dataset Profile	4
3. System Architecture and Methodology	5
3.1 Environment	5
4. Task 1 — HDFS + Spark Data Ingestion	5
5. Task 2 — Data Quality Engineering	5
5.1 Data Quality Issues Identified	6
5.2 Cleaning Treatment and Justification	6
6. Task 3 — Temporal Accident Intelligence	7
6.1 Time Period Definitions	7
6.2 Accidents by Time Period	7
6.3 Accidents by Day of Week	7
6.4 Weekday vs Weekend	7
6.5 Five Highest-Risk Time Periods (Time Period × Weekend/Weekday)	8
7. Task 4 — Accident Severity Index	8
7.1 Severity Score by Location (Top 10 Districts)	8
7.2 Severity Score by Road Type	9
7.3 Severity Score by Time Period	9
7.4 Why Highest Count ≠ Highest Risk	9
8. Task 5 — Dangerous-Factor Combination Analysis	10
8.1 Top 10 Combinations by Total Severity Burden	10
8.2 Top 10 Combinations by Average Severity (min. 30 occurrences)	10
9. Task 6 — Advanced Location Ranking with Window Functions	10
9.1 Top 3 Districts by Urban_or_Rural_Area	10
9.2 Top 3 Districts by Police Force (Sample)	11
10. Task 7 — Road Safety Risk Score	11
10.1 Weighting Justification	11
10.2 Top 15 Ranked Districts	12
11. Task 8 — Spark Execution and Performance Analysis	12
11.1 Transformations vs Actions	12
11.2 Shuffle Operations Identified	13
11.3 Stages and Tasks	13
11.4 Where cache() Could Improve Performance	13
12. Task 9 — Final Management Challenge: Five Road-Safety Priorities	14
Priority 1 — Target weekday commuting hours for enforcement and awareness	14

Priority 2 — Prioritise Birmingham as the top intervention location	14
Priority 3 — Target dangerous vehicle/road/time combinations, not just locations	14
Priority 4 — Invest in single carriageway road infrastructure	14
Priority 5 — Adopt the composite Risk Score as the standing prioritisation tool	15
13. Mandatory Spark Requirements — Compliance Summary	15
14. Conclusion	15
15. References	16

1. Introduction and Case Study

Rwanda National Police has reported road accidents as a significant road-safety concern. According to the NISR Statistical Yearbook 2024 (Table 14.2.6), Rwanda recorded 9,995 road accidents in 2023, including 761 fatal accidents, figures sourced directly from Rwanda National Police records.

This scale of loss motivates a data-driven approach to road-safety intelligence, rather than reliance on anecdotal or manually compiled reports. This project develops a prototype Rwanda Road Safety Intelligence System (RRSIS) that ingests, cleans, and analyses accident records to identify patterns that could support road-safety decision-making, using HDFS as the storage layer and Apache Spark (PySpark DataFrame) as the analytics engine.

1.1 Rwanda National Context (NISR Statistical Yearbook 2024)

Accident Category	2018	2019	2020	2021	2022	2023
Fatal accidents	597	673	675	621	676	761
Serious injury accidents	885	911	710	471	110	262
Minor injury accidents	1,887	1,485	1,326	3,688	5,186	5,037
Property damage accidents	2,242	1,584	1,492	3,859	4,362	3,935
Total	5,611	4,653	4,203	8,639	10,334	9,995

Table 1.1 — Rwanda road accidents by category, 2018–2023. Source: Rwanda National Police, via NISR Statistical Yearbook 2024, Table 14.2.6.

1.2 Objective

The system determines where, when, and under what circumstances accident risk is highest, using Spark DataFrame operations exclusively for all major analytical processing.

2. Dataset

This project uses the Kaggle “Road Accident Dataset” (xavierberge) as a surrogate dataset for developing the proposed RRSIS. This is UK-sourced accident data and does not represent actual Rwanda accident records; it is used solely to prototype and validate the analytics pipeline, per the assignment brief. Rwanda-specific context is drawn from the NISR and Rwanda National Police statistics presented in Section 1.1.

2.1 Dataset Profile

Property	Value
Source	Kaggle: Road Accident Dataset (xavierberge)
Raw records	307,973
Raw columns	23
Storage layer	HDFS (hdfs://localhost:9000/trsis/data/)

Property	Value
Processing engine	Apache Spark 3.5.1 (PySpark DataFrame API)
Granularity	One row per vehicle involved in an accident (not one row per accident)

Table 2.1 — Dataset profile.

3. System Architecture and Methodology

The processing pipeline follows the architecture specified in the project brief: raw data is stored in HDFS, loaded into Spark as a DataFrame, and passed through Cleaning, Analytics, and Risk Model stages to produce road-safety intelligence for management decisions.

Pipeline: Road accident data → HDFS (raw, 46-column-class dataset) → Apache Spark (PySpark DataFrame engine) → [Cleaning | Analytics | Risk Model] → Road Safety Intelligence → Management Decisions.

3.1 Environment

- Hadoop 3.3.6 (HDFS, single-node, Windows local install)
- Apache Spark 3.5.1 (PySpark), local[*] execution mode
- Java (OpenJDK) 11
- Jupyter Notebook for development and evidence capture

4. Task 1 — HDFS + Spark Data Ingestion

An HDFS directory was created and the dataset uploaded, then loaded directly into a PySpark DataFrame from HDFS.

```
hdfs dfs -mkdir -p /rrsis/data
hdfs dfs -put "Road Accident Data.csv" /rrsis/data/

df = spark.read.csv(
    "hdfs://localhost:9000/rrsis/data/Road Accident Data.csv",
    header=True, inferSchema=True)
```

The ingested dataset contains 307,973 records across 23 columns, including accident date/time, severity, location (district, latitude/longitude), road type, speed limit, weather and surface conditions, and vehicle type.

Metric	Value
Number of records	307,973
Number of columns	23
Storage path	hdfs://localhost:9000/rrsis/data/Road Accident Data.csv

Table 4.1 — Ingestion summary (Task 1 evidence).

5. Task 2 — Data Quality Engineering

Before analysis, the dataset was investigated for missing values, duplicates, inappropriate data types, and suspicious values. Seven distinct data-quality issues were identified, exceeding the minimum of five required.

5.1 Data Quality Issues Identified

#	Issue	Detail
1	Accident-level vs vehicle-level granularity	307,973 rows but only 197,644 distinct Accident_Index values — the dataset is recorded per vehicle involved, not per accident. Unaddressed, this double- or triple-counts multi-vehicle accidents in every downstream aggregation.
2	Time column mis-inferred as timestamp	Spark's inferSchema attached today's system date to every time value, since the source only contains a time-of-day. The date component is meaningless and must be discarded.
3	Accident Date stored as string	Prevented reliable extraction of month/weekday and required explicit parsing with to_date().
4	“None” used as a real category, not a missing marker	Carriageway_Hazards is coded “None” in 302,549 of 307,973 rows (~98%) — this represents “no hazard present,” a valid category, not missing data.
5	True missing values in contextual fields	Road_Surface_Conditions (317), Road_Type (1,534), Weather_Conditions (6,057), and Time (17) contained genuine nulls or empty strings.
6	Exact duplicate row	307,973 total rows vs 307,972 distinct rows — one full-row duplicate.
7	Rare Speed_limit categories	Only 3 rows at 10 mph and 2 at 15 mph — low-frequency values worth flagging as possible data-entry edge cases.

Table 5.1 — Data quality issues identified.

5.2 Cleaning Treatment and Justification

Per the assignment brief's caution against blind removal of missing records, each issue was treated individually rather than dropping all incomplete rows:

- Accident_Date_Parsed and Time_of_Day were derived via to_date() and date_format(), discarding only the corrupted date component of Time while preserving the true time-of-day.
- The single exact duplicate row was removed via dropDuplicates().
- The 17 rows with missing Time were dropped — a negligible loss (0.006% of records) that does not bias the temporal analysis.
- Road_Surface_Conditions, Road_Type, and Weather_Conditions nulls/blanks were imputed with the label “Unknown” rather than dropped, preserving the accident's casualty and severity information for Tasks 4–7 while flagging the specific field as unreliable.
- Carriageway_Hazards “None” values were left unchanged, since they represent a valid “no hazard” category rather than missing data — dropping or imputing them would have discarded a meaningful signal.
- A vehicle-level DataFrame (df_clean) and an accident-level DataFrame (df_accidents, deduplicated on Accident_Index) were both retained, so that vehicle-level detail remains available for Task 5 while Tasks 3, 4, 6, and 7 count unique accidents rather than vehicle-rows.

DataFrame	Row Count	Use
df_clean (vehicle-level)	307,955	Vehicle-type-sensitive analysis (Task 5 combinations)
df_accidents (accident-level)	197,644	Temporal, severity, location, and risk-score analysis (Tasks 3, 4, 6, 7)

Table 5.2 — Cleaned dataset row counts.

6. Task 3 — Temporal Accident Intelligence

Accidents were analysed by hour, day of week, month, and weekday vs weekend, using the accident-level DataFrame (df_accidents) so that multi-vehicle accidents are counted once.

6.1 Time Period Definitions

Five time periods were derived from the Hour field: Late Night (00:00–04:59), Morning (05:00–11:59), Afternoon (12:00–16:59), Evening (17:00–20:59), and Night (21:00–23:59).

6.2 Accidents by Time Period

Time Period	Accident Count
Afternoon	66,856
Morning	56,379
Evening	48,896
Night	15,799
Late Night	9,714

Table 6.1 — Accidents by time period.

6.3 Accidents by Day of Week

Day	Accident Count
Friday	32,376
Tuesday	29,853
Wednesday	29,850
Thursday	29,389
Monday	28,154
Saturday	26,544
Sunday	21,478

Table 6.2 — Accidents by day of week.

6.4 Weekday vs Weekend

Category	Accident Count
Weekday	149,622
Weekend	48,022

Table 6.3 — Weekday vs weekend split.

6.5 Five Highest-Risk Time Periods (Time Period × Weekend/Weekday)

Rank	Time Period	Day Type	Accident Count
1	Afternoon	Weekday	49,971
2	Morning	Weekday	45,953
3	Evening	Weekday	38,147
4	Afternoon	Weekend	16,885
5	Night	Weekday	11,226

Table 6.4 — Five highest-risk time periods.

All five highest-risk periods are weekday-dominated, indicating that traffic volume associated with commuting and work-hours activity is a stronger driver of accident frequency than weekend leisure driving. Peak single hours are 17:00 (17,134 accidents) and 16:00 (15,682 accidents), consistent with the evening commute.

7. Task 4 — Accident Severity Index

A severity-weighted score was computed as $\text{Severity Score} = \Sigma(\text{Number of accidents} \times \text{Severity Weight})$, using weights Slight = 1, Serious = 3, Fatal = 5.

Accident_Severity	Weight	Count
Slight	1	169,473
Serious	3	25,685
Fatal	5	2,486

Table 7.1 — Severity weight assignment.

7.1 Severity Score by Location (Top 10 Districts)

District	Accident Count	Severity Score
Birmingham	6,165	7,805
Westminster	2,811	4,341
Manchester	3,132	3,854
Sheffield	2,750	3,464
Liverpool	2,611	3,445
Cornwall	2,606	3,288

District	Accident Count	Severity Score
County Durham	2,228	2,904
Lambeth	2,250	2,884
Barnet	2,302	2,830
Southwark	1,977	2,579

Table 7.2 — Top 10 districts by total severity score.

7.2 Severity Score by Road Type

Road Type	Accident Count	Severity Score
Single carriageway	148,543	196,689
Dual carriageway	29,004	38,022
Roundabout	13,196	15,648
One way street	3,937	5,069
Slip road	1,951	2,297
Unknown	1,013	1,233

Table 7.3 — Severity score by road type.

7.3 Severity Score by Time Period

Time Period	Accident Count	Severity Score
Afternoon	66,856	86,082
Morning	56,379	72,461
Evening	48,896	63,872
Night	15,799	21,797
Late Night	9,714	14,746

Table 7.4 — Severity score by time period.

7.4 Why Highest Count ≠ Highest Risk

Birmingham records the highest total severity score (7,805) and highest accident count (6,165), but its average severity per accident is only $7,805 \div 6,165 \approx 1.27$ — dominated by Slight-weighted accidents occurring frequently. By contrast, districts such as South Northamptonshire (413 accidents, average severity 1.65), Daventry (395 accidents, average 1.57), and Purbeck (265 accidents, average 1.56) show a disproportionately higher share of Serious and Fatal outcomes per accident despite far lower volume.

This demonstrates that accident count measures exposure/frequency, while average severity measures outcome danger per incident. A location with few accidents that are consistently severe may indicate a specific dangerous road design or condition, and represents a distinct — and equally important — intervention priority from a high-volume, low-average-severity location like Birmingham. (Note: districts with very small accident counts, e.g. under 30, were treated with caution, since averages on small samples are statistically noisy.)

8. Task 5 — Dangerous-Factor Combination Analysis

Combinations of Road_Type, Speed_limit, Weather_Conditions, Time_Period, and Vehicle_Type were analysed using the vehicle-level DataFrame, since vehicle type is a vehicle-level attribute.

8.1 Top 10 Combinations by Total Severity Burden

Road Type	Speed	Weather	Period	Vehicle	Count	Severity
Single carriageway	30	Fine, no high winds	Afternoon	Car	25,659	32,487
Single carriageway	30	Fine, no high winds	Morning	Car	19,412	24,610
Single carriageway	30	Fine, no high winds	Evening	Car	17,807	22,993
Single carriageway	30	Fine, no high winds	Night	Car	5,345	7,295
Single carriageway	60	Fine, no high winds	Afternoon	Car	4,346	6,808

Table 8.1 — Top combinations by total severity burden (abridged; top 5 of 10 shown).

8.2 Top 10 Combinations by Average Severity (min. 30 occurrences)

Road Type	Speed	Weather	Period	Vehicle	Count	Avg Severity
Dual carriageway	70	Fine, no high winds	Late Night	Van ≤3.5t mgw	33	2.03
Single carriageway	60	Fine, no high winds	Late Night	Van ≤3.5t mgw	48	1.92
Single carriageway	40	Fine, no high winds	Late Night	Car	230	1.81
Single carriageway	60	Fine, no high winds	Evening	Motorcycle ≤125cc	78	1.79
Dual carriageway	70	Fine, no high winds	Late Night	Car	433	1.74

Table 8.2 — Top combinations by average severity, filtered to ≥30 occurrences (abridged; top 5 of 10 shown).

Filtering to combinations with at least 30 occurrences avoids a single freak fatal accident dominating the ranking. The most dangerous recurring pattern is commercial vans on high-speed dual carriageways at Late Night, averaging over 2.0 severity points per accident — higher than any car-only combination. Motorcycles also recur across several top-10 average-severity combinations, indicating elevated per-incident danger for two-wheeled vehicles independent of location.

9. Task 6 — Advanced Location Ranking with Window Functions

A dense_rank() Window function was used to rank districts by severity score within each geographical category, partitioned first by Urban_or_Rural_Area and separately by Police_Force. dense_rank() was chosen over row_number() because it preserves tied ranks — if two districts share an identical severity score, row_number() would arbitrarily drop one from the Top 3, while dense_rank() keeps both, which is more defensible for a road-safety prioritisation exercise.

9.1 Top 3 Districts by Urban_or_Rural_Area

Category	District	Accidents	Severity Score	Rank
Rural	Cornwall	1,992	2,584	1
Rural	County Durham	1,421	1,903	2
Rural	Wiltshire	1,158	1,816	3
Urban	Birmingham	6,099	7,717	1
Urban	Westminster	2,811	4,341	2
Urban	Manchester	3,072	3,772	3

Table 9.1 — Top 3 districts within each Urban/Rural category.

9.2 Top 3 Districts by Police Force (Sample)

Police Force	District	Accidents	Severity Score	Rank
Bedfordshire	Central Bedfordshire	1,225	1,691	1
Bedfordshire	Luton	998	1,224	2
Bedfordshire	Bedford	792	1,016	3
Devon and Cornwall	Cornwall	2,606	3,288	1
Devon and Cornwall	Plymouth	1,396	1,614	2
Greater Manchester	Manchester	3,132	3,854	1
Greater Manchester	Bolton	1,231	1,567	2

Table 9.2 — Top 3 districts within selected Police Force categories (full result covers all 30+ forces).

10. Task 7 — Road Safety Risk Score

A composite, data-derived Road Safety Risk Score was built from three normalised components at the district level:

- Accident frequency — total accident count per district (35% weight)
- Severity burden — total severity score per district (50% weight)
- Dangerous condition rate — share of accidents occurring in non-“Fine” weather or non-“Dry” road surface conditions (15% weight)

Each component was min-max normalised to a 0–1 range, then combined as: Risk Score = $(\text{Norm_Severity} \times 0.50) + (\text{Norm_Frequency} \times 0.35) + (\text{Norm_Danger} \times 0.15)$.

10.1 Weighting Justification

- Severity (50%) — reflects the primary purpose of the score: prioritising harm prevention, not just accident-count reduction.
- Frequency (35%) — high-frequency locations still merit priority even at lower average severity, since they represent greater total exposure and intervention opportunity.

- Dangerous conditions rate (15%) — a smaller weight, since this is a contextual risk factor rather than an outcome; it flags districts where infrastructure (drainage, lighting, surface quality) may be a contributing root cause.

10.2 Top 15 Ranked Districts

District	Accidents	Severity Score	Danger Rate	Risk Score
Birmingham	6,165	7,805	0.363	0.904
Manchester	3,132	3,854	0.397	0.484
Westminster	2,811	4,341	0.211	0.469
Sheffield	2,750	3,464	0.362	0.432
Liverpool	2,611	3,445	0.327	0.418
Cornwall	2,606	3,288	0.388	0.417
County Durham	2,228	2,904	0.400	0.373
Barnet	2,302	2,830	0.274	0.353
Lambeth	2,250	2,884	0.214	0.344
Doncaster	1,945	2,509	0.356	0.324
Nottingham	1,931	2,499	0.339	0.320
Southwark	1,977	2,579	0.236	0.313
Wiltshire	1,590	2,390	0.335	0.294
Camden	1,671	2,529	0.226	0.291
East Riding of Yorkshire	1,558	2,302	0.354	0.289

Table 10.1 — Top 15 districts by composite Road Safety Risk Score.

Birmingham's Risk Score (0.904) is nearly double the second-ranked district (Manchester, 0.484), confirming it as the clear top priority across all three components simultaneously rather than on frequency alone. This risk model is derived entirely from the data; no rankings were pre-assumed or manually assigned.

11. Task 8 — Spark Execution and Performance Analysis

`df.explain(True)` was used to inspect the logical and physical execution plans of key operations, identifying shuffle boundaries, stages, and caching opportunities.

11.1 Transformations vs Actions

All DataFrame operations used in this project — `withColumn`, `filter`, `groupBy`, `select`, `join` — are lazy transformations; no computation occurs until an action such as `.show()`, `.count()`, or `.collect()` is called. This laziness allows Spark's Catalyst optimizer to inspect and rewrite the full chain of operations (visible as the Optimized Logical Plan) before any execution begins.

11.2 Shuffle Operations Identified

The `groupBy("Local_Authority_(District)")` aggregation in Task 4 produces an Exchange `hashpartitioning(Local_Authority_(District), 200)` step in its physical plan. This shuffle is required because Spark must physically relocate all rows sharing the same district value onto a single executor before the count/sum aggregation can be computed — rows for “Birmingham,” for example, are scattered across partitions after the initial file scan and must be collected together.

The Task 6 Window function query produced three separate Exchange operators — more than the plain `groupBy` — because `dense_rank()` requires the data to be both partitioned by category (Urban_or_Rural_Area) and sorted within each partition by Severity_Score before ranking can be computed. This additional Sort step is why window functions are consistently more expensive than an equivalent plain aggregate.

A further finding: a simple `filter().select()` query on `df_clean` also triggered a shuffle, even though filter and select are narrow transformations on their own. This occurred because `df_clean` carries a `dropDuplicates()` call earlier in its lineage — a wide transformation that must shuffle all rows to detect duplicates. This demonstrates that shuffle cost is a property of a DataFrame's full lineage, not just the operator written in the current line of code.

The physical plan also showed Catalyst pushing the filter `Speed_limit > 30` down to the file scan itself (visible as `PushedFilters` in the `FileScan` node), reducing the volume of data read from HDFS before any Spark-side processing occurs.

11.3 Stages and Tasks

Each Exchange (shuffle) boundary marks a new stage in the Spark job. Within a stage, work is divided into tasks — one per partition — executed in parallel across the available executor cores. The Task 6 window-ranking query, with three Exchange operators, therefore spans four sequential stages.

11.4 Where `cache()` Could Improve Performance

`df_severity` (and its ancestors `df_time` and `df_accidents`) are reused across Tasks 4 through 7 — six or more separate `groupBy/aggregate` operations. Without caching, Spark recomputes the entire chain (HDFS read → deduplication → cleaning → column derivation) from scratch on every single downstream action, which is computationally wasteful for a DataFrame accessed this many times.

An isolated benchmark reusing `df_severity` across only two `groupBy` operations showed caching to be slower (5.07s) than no caching (3.76s), because the one-time cost of materialising the cache (a full read-through of ~198,000 rows) exceeded the savings from just two reuses, and `local[*]` execution has no network shuffle cost for caching to avoid. This is an expected result at small reuse counts: caching pays off only once its one-time materialisation cost is amortised across enough repeated actions. In the actual RRSIS pipeline, where `df_severity` underlies six or more aggregations across Tasks 4–7, the cumulative savings would be expected to outweigh the materialisation cost — caching benefits scale with the number of times a DataFrame is reused, not with any single access.

Scenario	Time (seconds)
Without cache (2 reuses)	3.76

Scenario	Time (seconds)
With cache (2 reuses, incl. materialisation)	5.07

Table 11.1 — Cache benchmark (2 reuses; caching only pays off with sufficient reuse).

12. Task 9 — Final Management Challenge: Five Road-Safety Priorities

Assuming road-safety authorities can prioritise only five intervention areas, the following recommendations follow directly from the Spark analysis above, each supported by numerical evidence.

Priority 1 — Target weekday commuting hours for enforcement and awareness

Data: df_time, accident-level records grouped by Time_Period × Weekend_Flag (Task 3).

Spark Analysis: groupBy("Time_Period", "Weekend_Flag").count().orderBy(desc).

Evidence: Weekday Afternoon (49,971), Weekday Morning (45,953), and Weekday Evening (38,147) are the three highest-volume time-period/day combinations nationally; hour-level data confirms 17:00 (17,134) and 16:00 (15,682) as the single worst hours.

Recommendation: Concentrate traffic police visibility and safety campaigns on weekday rush-hour windows (approx. 15:00–19:00 and the 08:00 peak).

Priority 2 — Prioritise Birmingham as the top intervention location

Data: location_components / Road_Safety_Risk_Score (Task 7).

Spark Analysis: Normalised composite score (50% severity, 35% frequency, 15% dangerous conditions).

Evidence: Birmingham scores 0.904 on the composite Risk Score — nearly double second-ranked Manchester (0.484) — and leads on every raw component (6,165 accidents; severity score 7,805).

Recommendation: Allocate the largest single share of the next intervention budget to Birmingham.

Priority 3 — Target dangerous vehicle/road/time combinations, not just locations

Data: combo_severity_filtered, combinations with ≥ 30 occurrences (Task 5).

Spark Analysis: groupBy(Road_Type, Speed_limit, Weather_Conditions, Time_Period, Vehicle_Type), ranked by average severity.

Evidence: The single worst recurring combination is commercial vans (≤ 3.5 t) on 70mph dual carriageways at Late Night, averaging 2.03 severity points per accident across 33 occurrences — the only combination above 2.0. Motorcycles recur repeatedly in the top 10.

Recommendation: Introduce night-time speed enforcement for commercial vehicles on dual carriageways and motorcycle-specific safety campaigns on 60mph single carriageways, applied nationally rather than location-by-location.

Priority 4 — Invest in single carriageway road infrastructure

Data: severity_by_roadtype (Task 4).

Spark Analysis: `groupBy("Road_Type").agg(count, sum(Severity_Weight)).`

Evidence: Single carriageways generated 196,689 severity points from 148,543 accidents — over five times the burden of the next category, dual carriageways (38,022 from 29,004 accidents).

Recommendation: Direct national road-engineering funding (lighting, junction redesign, speed-calming) primarily toward single carriageway roads.

Priority 5 — Adopt the composite Risk Score as the standing prioritisation tool

Data: `severity_by_location` average-severity comparison (Task 4) vs. `location_components` composite score (Task 7).

Spark Analysis: Contrast of `Severity_Score/Accident_Count` against the normalised weighted composite.

Evidence: South Northamptonshire ranks low by raw accident count (413) but shows the third-highest average severity nationally (1.65 per accident) — a pattern a simple count ranking would miss entirely.

Recommendation: Use the Task 7 composite Risk Score, not raw accident counts, to select each new round of intervention districts going forward.

13. Mandatory Spark Requirements — Compliance Summary

All major data processing and analytical results in this project were produced using PySpark DataFrame operations; Pandas was not used for core processing or analytical results.

Requirement	Where Demonstrated
<code>select() / filter() / where()</code>	Tasks 2, 8 (column selection, <code>Speed_limit</code> filtering)
<code>withColumn() / when()</code>	Tasks 2–7 (severity weights, time periods, weekend flag, cleaning)
<code>groupBy() / agg() / orderBy()</code>	Tasks 3–7 (all aggregation and ranking)
Window functions	Task 6 (<code>dense_rank()</code> partitioned ranking)
Advanced operation 1: <code>join()</code>	Task 7 (merging frequency, severity, and danger-rate components)
Advanced operation 2: <code>dropDuplicates()</code> with subset keys	Task 2 (accident-level deduplication)
Advanced operation 3: <code>explain(True)</code> plan inspection	Task 8 (execution plan analysis)
Read from HDFS	Task 1 (<code>spark.read.csv</code> on <code>hdfs://</code> path)

Table 13.1 — Mandatory Spark requirement compliance.

14. Conclusion

This project developed a working prototype Rwanda Road Safety Intelligence System using HDFS for storage and Apache Spark (PySpark DataFrames) as the sole analytics engine, satisfying all ten technical tasks specified in the project brief. Using a UK-sourced surrogate dataset of 307,973 vehicle-involvement records (197,644 unique accidents), the system identified clear temporal risk concentrations (weekday

afternoons and the evening commute), a top-priority location (Birmingham) via a data-derived composite risk score, dangerous factor combinations (commercial vans on high-speed dual carriageways at night), and a principled distinction between accident frequency and accident severity that a naive count-based ranking would miss. These findings translate directly into five evidence-based intervention priorities for road-safety authorities, and the same pipeline architecture is directly transferable to genuine Rwanda National Police accident records when they become available in a comparable schema.

15. References

National Institute of Statistics of Rwanda (NISR). Statistical Yearbook 2024, Table 14.2.6 — Road Accidents. Source data: Rwanda National Police.

Kaggle. Road Accident Dataset (xavierberge). <https://www.kaggle.com/datasets/xavierberge/road-accident-dataset/data>

Apache Spark Documentation. <https://spark.apache.org/docs/latest/>

Apache Hadoop Documentation. <https://hadoop.apache.org/docs/stable/>